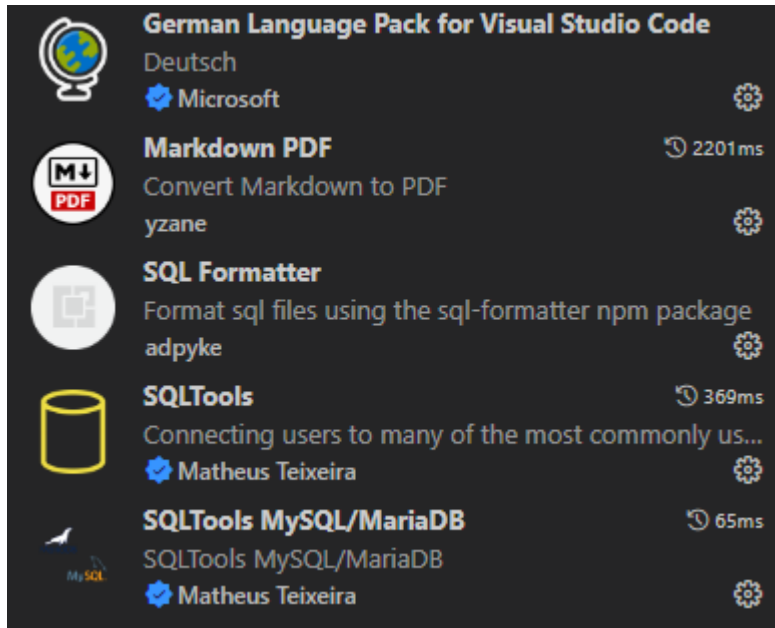


mariadb cheatsheet



SYNTAX

EXAMPLE

Tabelle erstellen

```
CREATE TABLE `zip` (  
  `id` INT UNSIGNED NOT NULL AUTO_INCREMENT, -- bigint UNSIGNED NOT NULL  
  auto_increment  
  `zip` SMALLINT(4) UNSIGNED NOT NULL,  
  `city` VARCHAR(255) NOT NULL,  
  
  PRIMARY KEY(`id`),  
  UNIQUE(`zip`, `city`)  
);  
  
CREATE TABLE `customer` (  
  `id` INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  `fk_zip_id` INT UNSIGNED NOT NULL,  
  `firstname` VARCHAR(255) NOT NULL,  
  `lastname` VARCHAR(255) NOT NULL,  
  `address` VARCHAR(255) NOT NULL,  
  `email` VARCHAR(384) NOT NULL UNIQUE,  
  `password` VARCHAR(255) NOT NULL,  
  `phone` VARCHAR(255),  
  
  PRIMARY KEY(`id`),  
  FOREIGN KEY(`fk_zip_id`) REFERENCES `zip`(`id`) ON UPDATE CASCADE ON DELETE  
  CASCADE  
);
```

Zwischentabelle erstellen

SYNTAX

```
CREATE TABLE `table_name` (  
  `fk_name_id` INT NOT NULL,  
  `fk_name_id` INT NOT NULL,  
  FOREIGN KEY (`fk_order_id`) REFERENCES `order`(`id`),  
  FOREIGN KEY (`fk_product_id`) REFERENCES `product`(`id`)  
);
```

EXAMPLE

```
CREATE TABLE `order_has_product` (  
  `fk_order_id` INT NOT NULL,  
  `fk_product_id` INT NOT NULL,  
  FOREIGN KEY (`fk_name_id`) REFERENCES `table_name`(`id`),  
  FOREIGN KEY (`fk_name_id`) REFERENCES `table_name`(`id`)  
);
```

INSERT

SYNTAX

```
INSERT INTO table_name (column1, column2, column3, ...)  
VALUES (value1, value2, value3, ...);
```

EXAMPLE

```
INSERT INTO Customers (CustomerName, ContactName, Address, City, PostalCode,  
Country)  
VALUES ('Cardinal', 'Tom B. Erichsen', 'Skagen 21', 'Stavanger', '4006',  
'Norway');
```

ALTER

SYNTAX

```
ALTER TABLE table_name MODIFY column_name datatype;
```

EXAMPLE

```
ALTER TABLE customers MODIFY birthdate DATE;
```

SELECT mit COUNT();

SYNTAX

```
SELECT COUNT(column) FROM table;
```

EXAMPLE

```
SELECT COUNT(*) FROM orders;
```

INNER JOIN

SYNTAX

```
SELECT column1, column2, ...  
FROM table1  
INNER JOIN table2  
ON table1.column = table2.column;
```

EXAMPLE

```
SELECT customers.name, orders.order_id  
FROM customers  
INNER JOIN orders  
ON customers.customer_id = orders.customer_id;
```

LEFT JOIN

SYNTAX

```
SELECT column1, column2, ...  
FROM table1  
LEFT JOIN table2  
ON table1.column = table2.column;
```

EXAMPLE

```
SELECT customer.firstname, order_entry.ordered_at  
FROM customer  
LEFT JOIN order_entry  
ON customer.id = order_entry.fk_customer_id;
```

RIGHT JOIN**SYNTAX**

```
SELECT column1, column2, ...  
FROM table1  
RIGHT JOIN table2  
ON table1.column = table2.column;
```

EXAMPLE

```
SELECT customer.firstname, order_entry.ordered_at  
FROM customer  
RIGHT JOIN order_entry  
ON customer.id = order_entry.fk_customer_id;
```

SUBQUERY

```
SELECT ordered_at, delivered_at, id FROM order_entry WHERE id > (SELECT COUNT(id)  
FROM order_entry);
```

Erklärung

SELECT reihe/attribut: ordered_at, delivered_at, id
VON tabelle: order_entry
WO die ID = SELECT COUNT(id) FROM order_entry = (22)

DELETE

SYNTAX

```
DELETE FROM table_name WHERE condition;
```

EXAMPLE

```
DELETE FROM Customers WHERE CustomerName='Alfreds Futterkiste';
```

UPDATE

SYNTAX

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

EXAMPLE

```
UPDATE Customers  
SET ContactName = 'Alfred Schmidt', City= 'Frankfurt'  
WHERE CustomerID = 1;
```

SELECT JOIN mit GROUP BY & ORDER BY

SYNTAX

```
SELECT column1, aggregate_function(column2)  
FROM table1  
JOIN table2  
ON table1.column = table2.column
```

```
GROUP BY column1
ORDER BY aggregate_function(column2) DESC/ASC;
```

EXAMPLE

```
SELECT customers.country, SUM(orders.total)
FROM customers
JOIN orders
ON customers.customer_id = orders.customer_id
GROUP BY customers.country
ORDER BY SUM(orders.total) DESC;
```

GROUP BY & ORDER BY**SYNTAX**

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
GROUP BY column_name;
```

EXAMPLE

```
SELECT department, SUM(salary)
FROM employees
GROUP BY department;
```

SYNTAX

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
GROUP BY column_name;
```

EXAMPLE

```
SELECT first_name, last_name, salary
FROM employees
ORDER BY salary DESC;
```

EXAMPLE

```
SELECT department, SUM(salary)
FROM employees
GROUP BY department;
```

SELECT (MIT FUNCTIONS)**SYNTAX**

```
SELECT column1, column2, ...
FROM table_name;
```

EXAMPLE

```
SELECT CustomerName, City FROM Customers;
```

LIKE (MIT FUNCTIONS)**SYNTAX**

```
SELECT column1, column2, ...
FROM table_name
WHERE columnN LIKE pattern;
```

EXAMPLE

```
SELECT * FROM Customers
WHERE CustomerName LIKE 'a%';
```

INDICES / INDEX**SYNTAX**

```
CREATE (UNIQUE) INDEX index_name  
ON table_name (column1, column2, ...);
```

EXAMPLE

```
CREATE INDEX idx_pname  
ON Persons (LastName, FirstName);
```

LOAD**SYNTAX**

```
LOAD DATA INFILE '/path/to/data.csv'  
INTO TABLE customers  
FIELDS TERMINATED BY ','  
ENCLOSED BY '"'  
LINES TERMINATED BY '\n'  
IGNORE 1 ROWS  
(customer_id, first_name, last_name, email);
```

GRANT**SYNTAX**

```
GRANT EXECUTE ON TestProc TO TesterRole WITH GRANT OPTION;  
EXEC sp_addrolemember TesterRole, User1;
```

MAX()**SYNTAX**

```
SELECT MAX(column_name)  
FROM table_name  
WHERE condition;
```

MIN()

SYNTAX

```
SELECT MIN(column_name)
FROM table_name
WHERE condition;
```

COUNT()**SYNTAX**

```
SELECT COUNT(column_name)
FROM table_name
WHERE condition;
```

AVG()**SYNTAX**

```
SELECT AVG(column_name)
FROM table_name
WHERE condition;
```

SUM()**SYNTAX**

```
SELECT SUM(column_name)
FROM table_name
WHERE condition;
```

CONCAT()**SYNTAX**

```
select concat(' Hello, ', 'my name is ', 'Paul');
```

SQRT()

SYNTAX

```
SELECT SQRT(64);
```

...

```
SELECT person.firstname AS "Vorname", person.lastname AS "Nachname",  
person_course_execution.fk_course_execution_id AS "Anzahl Anmeldungen"  
FROM person  
JOIN person_course_execution on person.id =  
person_course_execution.fk_participant_id  
GROUP BY firstname = "Ernst", "Heinz", lastname = "Lorbeer"  
ORDER BY person_course_execution.fk_course_execution_id DESC;
```

Prüfungsvorbereitungen Lösungen

```
CREATE TABLE category (  
  id INT AUTO_INCREMENT,  
  name VARCHAR(255) NOT NULL,  
  PRIMARY KEY(id)  
);  
  
CREATE TABLE category_course (  
  fk_category_id INT NOT NULL,  
  fk_course_id INT NOT NULL,  
  created_at DATETIME NOT NULL,  
  FOREIGN KEY (fk_category_id) REFERENCES category(id),  
  FOREIGN KEY (fk_course_id) REFERENCES course(id)  
);  
  
ALTER TABLE category_course MODIFY created_at DATETIME NOT NULL DEFAULT NOW();  
  
INSERT INTO category (name) VALUES  
  ("Applikationsentwicklung"),  
  ("Datenbanken"),  
  ("MANAGEMENT"),  
  ("Netzwerk");  
  
INSERT INTO category_course (fk_category_id, fk_course_id) VALUES (2,2);  
  
DELETE FROM person_course_execution WHERE fk_participant_id = 14;
```

```
DELETE FROM person WHERE id = 14;

UPDATE person SET lastname = "Nuesch" WHERE firstname = "Patrick" AND lastname = "Wirz";

SELECT p.firstname, p.lastname, c.name
FROM
  person p
  INNER JOIN person_course_execution pce ON p.id = pce.fk_participant_id
  INNER JOIN course_execution ce ON pce.fk_course_execution_id = ce.id
  INNER JOIN course c ON ce.fk_course_id = c.id
WHERE ce.room = 'Red';

SELECT p.firstname as "Vorname", p.lastname as "Nachname",
COUNT(pce.fk_participant_id) as "Anzahl Anmeldungen"
FROM
  person p
  INNER JOIN person_course_execution pce ON p.id = pce.fk_participant_id
GROUP BY p.id
HAVING COUNT(pce.fk_participant_id) <= 2
ORDER BY COUNT(pce.fk_participant_id) DESC;
```